

Week 3 Assignment

Name: Labhade Siddharth Popat

Student ID: CT_CSI_DE_1176

Objective:

Use Subqueries, CTEs, and Window Functions to analyze sales data from the Superstore dataset.

Tasks

Step 1: Setup Data

1. Import the Superstore dataset into a table named superstore_raw.

Output:

The top screenshot shows the MySQL Workbench interface with the following SQL script:

```
28 SHOW VARIABLES LIKE 'local_infile';
29 SET GLOBAL local_infile = 1;
30
31 LOAD DATA LOCAL INFILE 'C:/Sample - Superstore.csv' INTO TABLE superstore_raw;
32
33 CHARACTER SET latin1;
34
35 LINES TERMINATED BY '\n';
36
37 ENCLOSED BY ',';
38
39 LINES TERMINATED BY '\n';
40
41 EXPORT -I ROWS;
42
43 SELECT COUNT(*)
44 FROM superstore_raw;
45
46 SELECT *
47 FROM superstore_raw;
```

The bottom screenshot shows the MySQL Workbench interface with the following SQL script:

```
1 create database superstore_db;
2 use superstore_db;
3
4 show tables;
5
6 SELECT *
7 FROM superstore_raw
8 LIMIT 20;
```

The bottom screenshot also displays a table of data with the following columns: Row ID, Order ID, Order Date, Ship Date, Ship Mode, Customer ID, Customer Name, Segment, Country, City, State, Postal Code, Region, and Product. The table contains 6 rows of data.

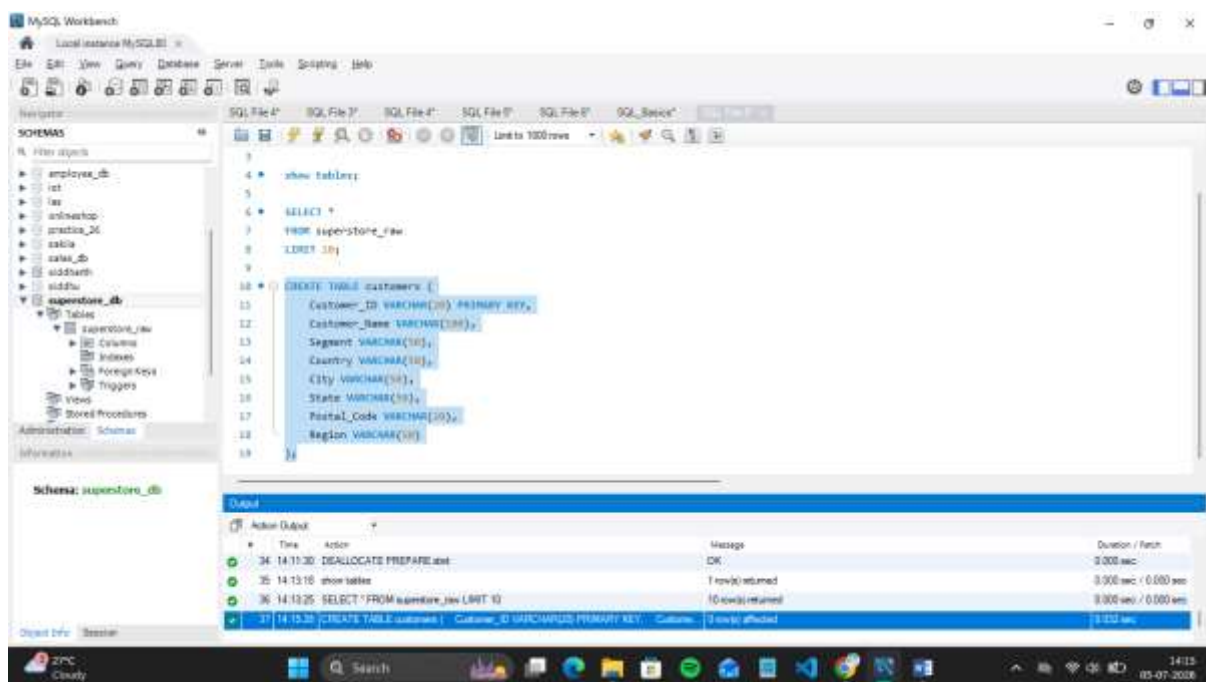
2. Create these 3 tables from it:

a. customers Table :

SQL Query:

```
CREATE TABLE customers (  
    Customer_ID VARCHAR(20) PRIMARY KEY,  
    Customer_Name VARCHAR(100),  
    Segment VARCHAR(50),  
    Country VARCHAR(50),  
    City VARCHAR(50),  
    State VARCHAR(50),  
    Postal_Code VARCHAR(20),  
    Region VARCHAR(50)  
);
```

Output:

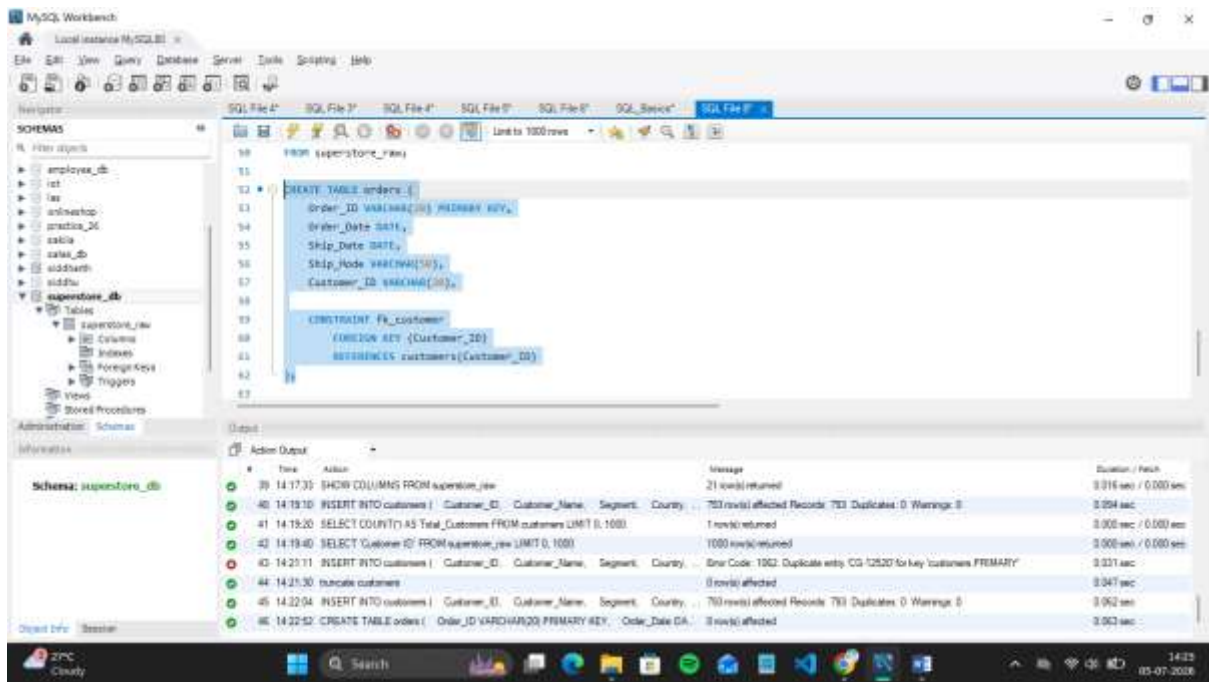


b. orders Table :

SQL Query:

```
CREATE TABLE orders (  
    Order_ID VARCHAR(20) PRIMARY KEY,  
    Order_Date DATE,  
    Ship_Date DATE,  
    Ship_Mode VARCHAR(50),  
    Customer_ID VARCHAR(20),  
  
    CONSTRAINT fk_customer  
        FOREIGN KEY (Customer_ID)  
        REFERENCES customers(Customer_ID)  
);
```

Output:



c. products Table:

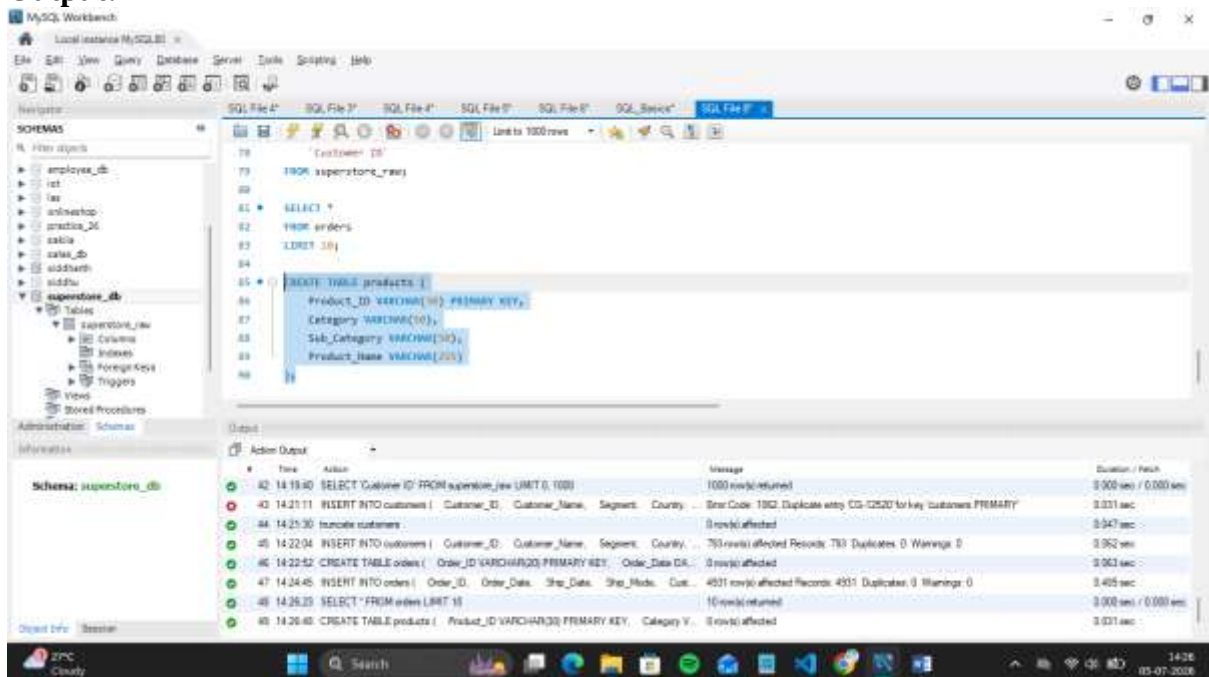
SQL Query:

```

CREATE TABLE products (
  Product_ID VARCHAR(30) PRIMARY KEY,
  Category VARCHAR(50),
  Sub_Category VARCHAR(50),
  Product_Name VARCHAR(255)
);

```

Output:



3. Insert data into these tables using SELECT DISTINCT.

a. Insert data into customers

SQL Query:

```
INSERT INTO customers
(
    Customer_ID,
    Customer_Name,
    Segment,
    Country,
    City,
    State,
    Postal_Code,
    Region
)
SELECT
    `Customer ID`,
    MAX(`Customer Name`),
    MAX(`Segment`),
    MAX(`Country`),
    MAX(`City`),
    MAX(`State`),
    MAX(`Postal Code`),
    MAX(`Region`)
FROM superstore_raw
GROUP BY `Customer ID`;
```

Output:

The screenshot displays the MySQL Workbench interface. On the left, the 'SCHEMAS' pane shows the 'superstore_db' database selected. The main editor window contains the following SQL query:

```
INSERT INTO customers
(
    Customer_ID,
    Customer_Name,
    Segment,
    Country,
    City,
    State,
    Postal_Code,
    Region
)
SELECT
    `Customer ID`,
    MAX(`Customer Name`),
    MAX(`Segment`),
    MAX(`Country`),
    MAX(`City`),
    MAX(`State`),
    MAX(`Postal Code`),
    MAX(`Region`)
FROM superstore_raw
GROUP BY `Customer ID`;
```

The 'Output' pane at the bottom shows the execution results of the query. It includes a table with columns: #, Time, Action, Message, and Duration / Patch. The results are as follows:

#	Time	Action	Message	Duration / Patch
39	14:16:33	INSERT INTO customers SELECT `Customer ID`, MAX(`Customer Name`), MAX(`Segment`), MAX(`Country`), MAX(`City`), MAX(`State`), MAX(`Postal Code`), MAX(`Region`)	Error Code: 1054. Unknown column 'Customer_ID' in field list	0.000 sec
39	14:17:23	SHOW COLUMNS FROM superstore_raw	21 row(s) returned	0.018 sec / 0.000 sec
40	14:18:10	INSERT INTO customers (`Customer_ID`, `Customer_Name`, `Segment`, `Country`, `City`, `State`, `Postal_Code`, `Region`)	793 row(s) affected Records: 793 Duplicates: 0 Warnings: 0	0.094 sec
41	14:19:20	SELECT COUNT(*) AS Total_Customers FROM customers LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
42	14:19:40	SELECT `Customer ID` FROM superstore_raw LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec
43	14:21:11	INSERT INTO customers (`Customer_ID`, `Customer_Name`, `Segment`, `Country`, `City`, `State`, `Postal_Code`, `Region`)	Error Code: 1062. Duplicate entry 'CG-12537' for key 'customers.PRIMARY'	0.031 sec
44	14:21:30	truncate customers	0 row(s) affected	0.047 sec
45	14:22:04	INSERT INTO customers (`Customer_ID`, `Customer_Name`, `Segment`, `Country`, `City`, `State`, `Postal_Code`, `Region`)	793 row(s) affected Records: 793 Duplicates: 0 Warnings: 0	0.062 sec

b. Insert into orders Table:

SQL Query:

```
INSERT INTO orders
```

```
(
```

```
Order_ID,
```

```
Order_Date,
```

```
Ship_Date,
```

```
Ship_Mode,
```

```
Customer_ID
```

```
)
```

```
SELECT DISTINCT
```

```
`Order ID`,
```

```
STR_TO_DATE(`Order Date`, '%m/%d/%Y'),
```

```
STR_TO_DATE(`Ship Date`, '%m/%d/%Y'),
```

```
`Ship Mode`,
```

```
`Customer ID`
```

```
FROM superstore_raw;
```

Output:

The screenshot displays the MySQL Workbench interface. The left sidebar shows the 'SCHEMAS' tree with 'superstore_db' selected. The main editor window contains the following SQL queries:

```
INSERT INTO orders
(
  Order_ID,
  Order_Date,
  Ship_Date,
  Ship_Mode,
  Customer_ID
)
SELECT DISTINCT
  `Order ID`,
  STR_TO_DATE(`Order Date`, '%m/%d/%Y'),
  STR_TO_DATE(`Ship Date`, '%m/%d/%Y'),
  `Ship Mode`,
  `Customer ID`
FROM superstore_raw;
```

The 'Output' tab at the bottom shows the results of the queries. The table has columns: #, Time, Action, Message, and Duration / Patch. The messages indicate the successful execution of the INSERT and SELECT queries, with 1000 rows affected in the INSERT statement.

#	Time	Action	Message	Duration / Patch
40	14:19:10	INSERT INTO customers (Customer_ID, Customer_Name, Segment, Country, ...	793 rows affected Records: 793 Duplicates: 0 Warnings: 0	0.004 sec
41	14:19:20	SELECT COUNT(*) AS Total_Customers FROM customers LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
42	14:19:40	SELECT Customer_ID FROM superstore_raw LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec
43	14:21:11	INSERT INTO customers (Customer_ID, Customer_Name, Segment, Country, ...	Error Code: 1062, Duplicate entry 'CG-12520' for key 'customers.PRIMARY'	0.021 sec
44	14:21:30	truncate customers	0 row(s) affected	0.047 sec
45	14:22:04	INSERT INTO customers (Customer_ID, Customer_Name, Segment, Country, ...	793 rows affected Records: 793 Duplicates: 0 Warnings: 0	0.063 sec
46	14:22:52	CREATE TABLE orders (Order_ID VARCHAR(20) PRIMARY KEY, Order_Date DA...	0 row(s) affected	0.063 sec
47	14:24:45	INSERT INTO orders (Order_ID, Order_Date, Ship_Date, Ship_Mode, Cust...	4921 row(s) affected Records: 4921 Duplicates: 0 Warnings: 0	0.408 sec

c. **Insert into products table:**

SQL Query:

```
INSERT INTO products
(
    Product_ID,
    Category,
    Sub_Category,
    Product_Name
)
SELECT
    `Product ID`,
    MAX(`Category`),
    MAX(`Sub-Category`),
    MAX(`Product Name`)
FROM superstore_raw
GROUP BY `Product ID`;
```

Output:

The screenshot displays the MySQL Workbench interface. The left sidebar shows the 'SCHEMAS' panel with a tree view of databases, including 'superstore_db'. The central editor window contains the following SQL query:

```
INSERT INTO products
(
    Product_ID,
    Category,
    Sub_Category,
    Product_Name
)
SELECT
    `Product ID`,
    MAX(`Category`),
    MAX(`Sub-Category`),
    MAX(`Product Name`)
FROM superstore_raw
GROUP BY `Product ID`;
```

The bottom panel shows the 'Action Output' tab, which lists the execution results of the query. The output is as follows:

#	Time	Action	Message	Duration / Fetch
45	14:22:34	INSERT INTO customers Customer_ID, Customer_Name, Segment, Country, ...	763 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.362 sec
46	14:22:52	CREATE TABLE orders Order_ID VARCHAR(25) PRIMARY KEY, Order_Date DATE, ...	0 row(s) affected	0.063 sec
47	14:24:45	INSERT INTO orders Order_ID, Order_Date, Ship_Date, Ship_Mode, Cust...	4931 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.426 sec
48	14:26:23	SELECT * FROM orders LIMIT 10	10 row(s) returned	0.006 sec / 0.000 sec
49	14:26:40	CREATE TABLE products Product_ID VARCHAR(10) PRIMARY KEY, Category VARCHAR(25), ...	0 row(s) affected	0.021 sec
50	14:29:16	INSERT INTO products Product_ID, Category, Sub_Category, Product_Name, ...	Error Code: 1062. Duplicate entry 'FURN-CH-1000148' for key 'products.PRIMARY'	0.047 sec
51	14:30:17	SELECT COUNT(*) FROM products LIMIT 3, 1000	1 row(s) returned	0.000 sec / 0.000 sec
52	14:30:41	INSERT INTO products Product_ID, Category, Sub_Category, Product_Name, ...	1612 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.148 sec

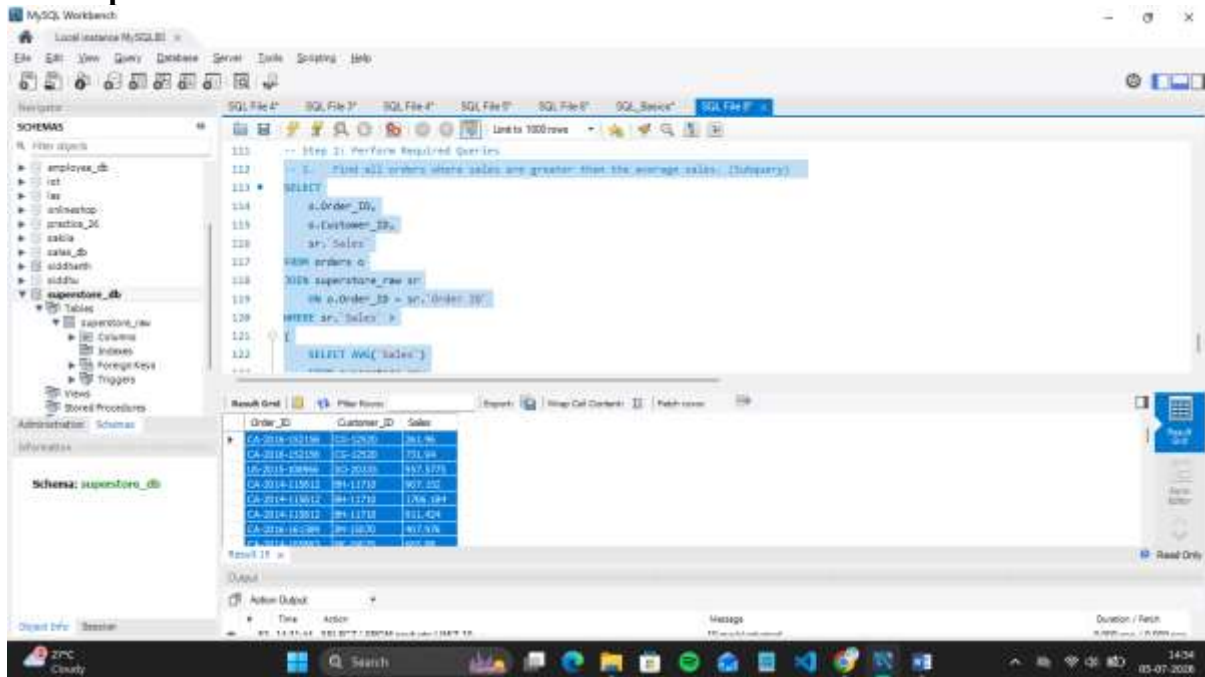
Step 2: Perform Required Queries

1. Find all orders where sales are greater than the average sales. (Subquery)

SQL Query:

```
SELECT
    o.Order_ID,
    o.Customer_ID,
    sr.`Sales`
FROM orders o
JOIN superstore_raw sr
    ON o.Order_ID = sr.`Order ID`
WHERE sr.`Sales` >
(
    SELECT AVG(`Sales`)
    FROM superstore_raw
);
```

Output:



The screenshot shows the MySQL Workbench interface. The SQL editor contains the query for finding orders with sales greater than the average. The output is displayed in a table with columns Order_ID, Customer_ID, and Sales.

Order_ID	Customer_ID	Sales
CA-2018-051196	105-52920	361.96
CA-2018-051196	105-52920	751.96
US-2018-098966	105-20105	453.1779
CA-2018-115011	104-11710	907.002
CA-2018-115011	104-11710	1706.194
CA-2018-115011	104-11710	811.424
CA-2018-161086	101-10120	917.576
CA-2018-161086	101-10120	600.88

2. Find the highest sales order for each customer. (Subquery)

SQL Query:

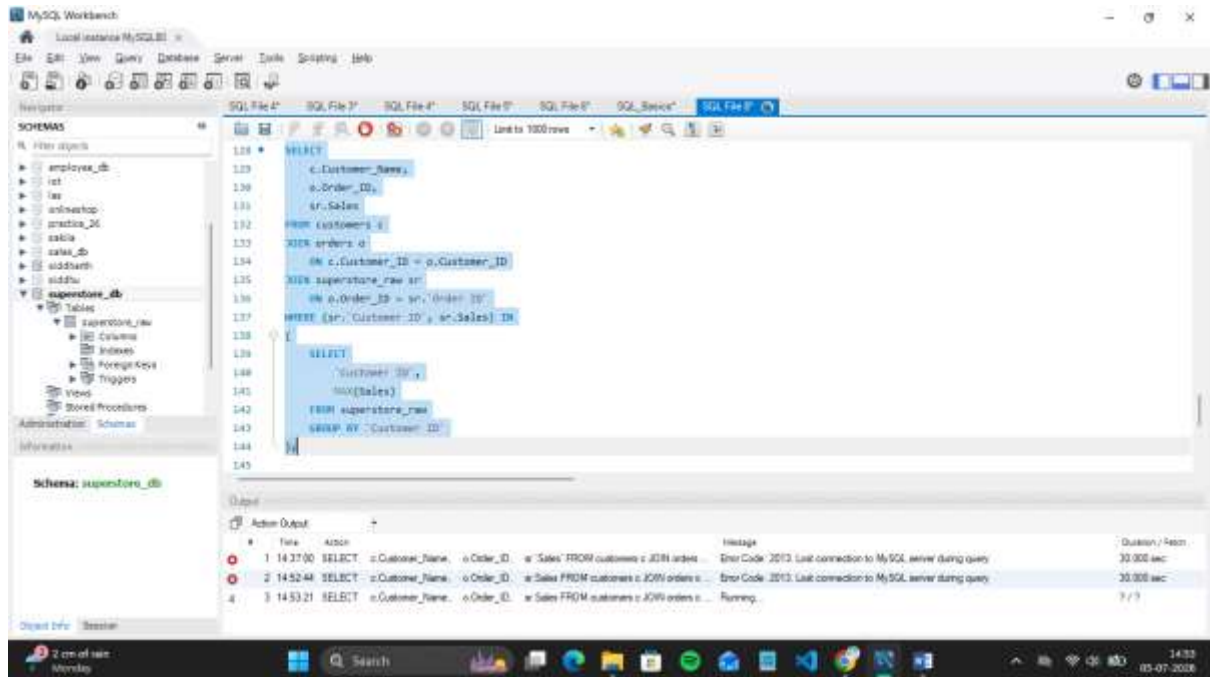
```
SELECT
    c.Customer_Name,
    o.Order_ID,
    sr.Sales
FROM customers c
JOIN orders o
    ON c.Customer_ID = o.Customer_ID
JOIN superstore_raw sr
    ON o.Order_ID = sr.`Order ID`
```

```

WHERE (sr.`Customer ID`, sr.Sales) IN
(
    SELECT
        `Customer ID`,
        MAX(Sales)
    FROM superstore_raw
    GROUP BY `Customer ID`
);

```

Output:



3. Calculate total sales for each customer. (CTE)

SQL Query:

```

WITH CustomerSales AS
(
    SELECT
        c.Customer_ID,
        c.Customer_Name,
        SUM(sr.Sales) AS Total_Sales
    FROM customers c
    JOIN orders o
        ON c.Customer_ID = o.Customer_ID
    JOIN superstore_raw sr
        ON o.Order_ID = sr.`Order ID`
    GROUP BY
        c.Customer_ID,
        c.Customer_Name

```



```

SELECT *
FROM CustomerSales
ORDER BY Total_Sales DESC;

```

Output:

The screenshot shows the MySQL Workbench interface. The SQL editor contains a query that uses a CTE to calculate total sales for each customer. The results grid displays the output of this query, showing customer IDs, names, and their total sales.

Customer_ID	Customer_Name	Total_Sales
94-20-020	Seoul-Hill	15963.25
15-20-005	Tampa-Channel	15017.24
42-20-000	Sanford-Bush	15117.235
19-21-005	Tam-Alderson	14585.63
40-20-025	Adrian-Barber	14105.630000000007
50-20-005	Saint-Channel	14142.11
61-20-005	Sanford-Bush	14074.537

4. Find customers whose total sales are above average. (CTE + Subquery)

SQL Query:

```

WITH CustomerSales AS
(
    SELECT
        c.Customer_ID,
        c.Customer_Name,
        SUM(sr.Sales) AS Total_Sales
    FROM customers c
    JOIN orders o
        ON c.Customer_ID = o.Customer_ID
    JOIN superstore_raw sr
        ON o.Order_ID = sr.`Order ID`
    GROUP BY
        c.Customer_ID,
        c.Customer_Name
)

SELECT
    Customer_ID,

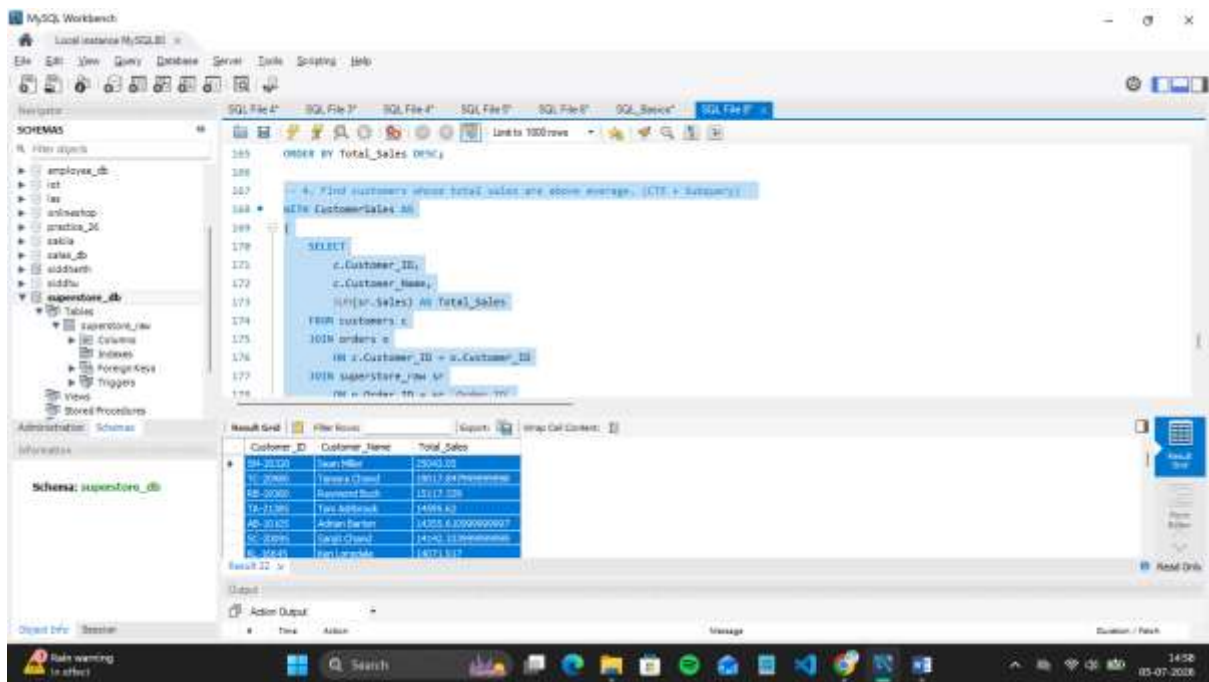
```

```

Customer_Name,
Total_Sales
FROM CustomerSales
WHERE Total_Sales >
(
    SELECT AVG(Total_Sales)
    FROM CustomerSales
)
ORDER BY Total_Sales DESC;

```

Output:



The screenshot shows the MySQL Workbench interface. The SQL editor contains a query to find customers whose total sales are above the average. The results grid displays the following data:

Customer_ID	Customer_Name	Total_Sales
54-25120	Seur-Niel	25040.00
11-20760	Yareka (Lionel)	18017.84
45-30900	Raymond Bush	15117.526
19-21385	Yari delacruz	74485.62
46-20400	Adrian Barrios	74125.6
20-32895	David (David)	74125.6
6-12245	Reynaldo	74125.6

5. Rank all customers based on total sales. (Window Function)

SQL Query:

```

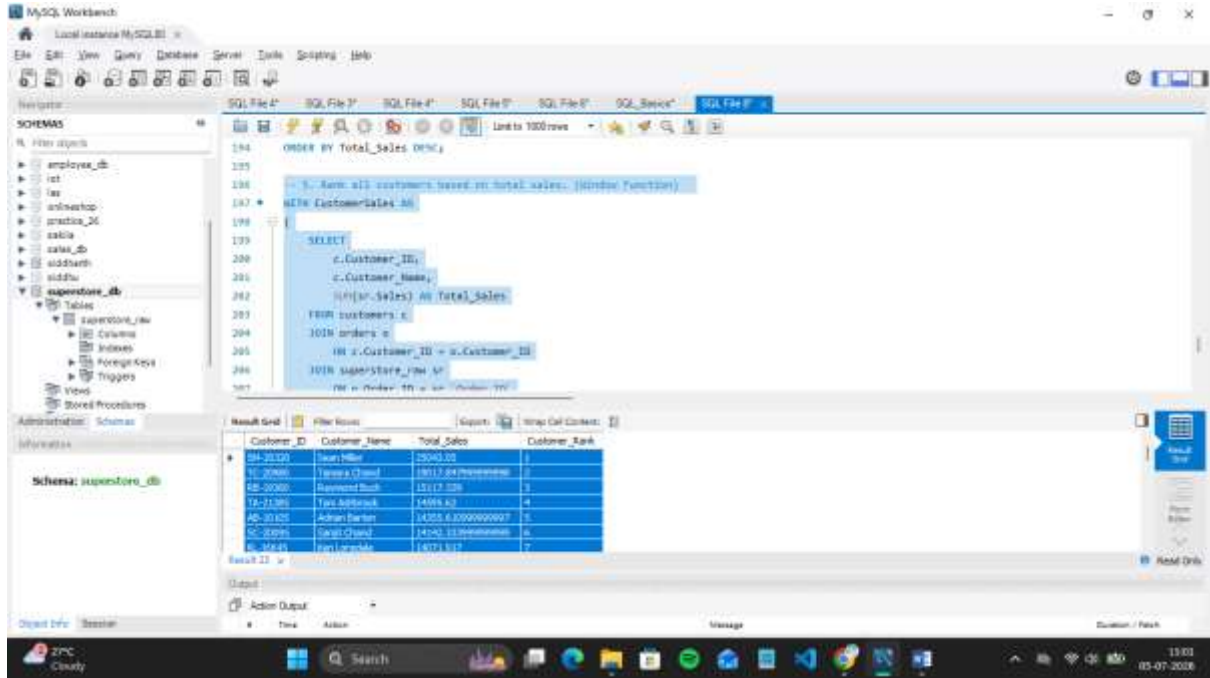
WITH CustomerSales AS
(
    SELECT
        c.Customer_ID,
        c.Customer_Name,
        SUM(sr.Sales) AS Total_Sales
    FROM customers c
    JOIN orders o
        ON c.Customer_ID = o.Customer_ID
    JOIN superstore_raw sr
        ON o.Order_ID = sr.`Order ID`
    GROUP BY
        c.Customer_ID,
        c.Customer_Name
)

```

)

```
SELECT
    Customer_ID,
    Customer_Name,
    Total_Sales,
    RANK() OVER (ORDER BY Total_Sales DESC) AS Customer_Rank
FROM CustomerSales
ORDER BY Customer_Rank;
```

Output:



The screenshot shows the MySQL Workbench interface. The SQL editor contains a query that ranks customers by total sales. The output is displayed in a table with the following data:

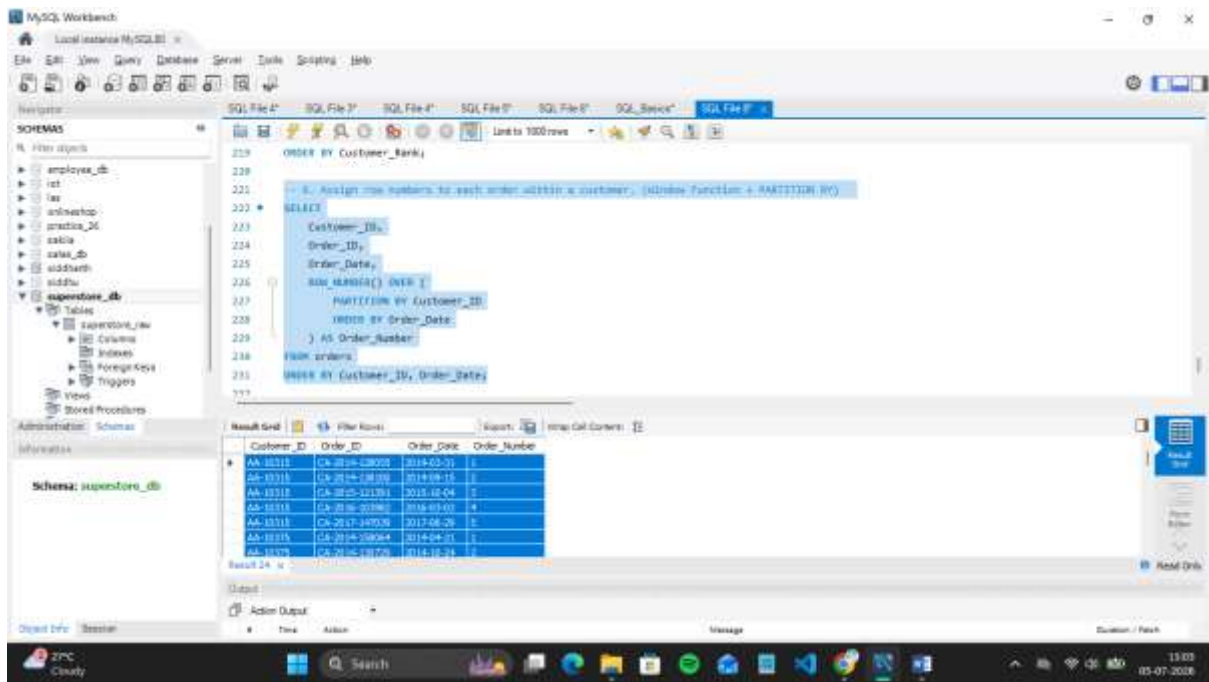
Customer_ID	Customer_Name	Total_Sales	Customer_Rank
94-20320	Seun Hwa	25040.00	1
10-20886	Tamara Choud	18617.84	2
42-20260	Baronard Bush	15117.00	3
16-21381	Tara Adhewak	14086.62	4
40-20425	Adnan Barton	14055.60	5
00-20095	Savit Choud	14042.11	6
00-20426	Ignacio Sola	13024.00	7

6. Assign row numbers to each order within a customer. (Window Function + PARTITION BY)

SQL Query:

```
SELECT
    Customer_ID,
    Order_ID,
    Order_Date,
    ROW_NUMBER() OVER (
        PARTITION BY Customer_ID
        ORDER BY Order_Date
    ) AS Order_Number
FROM orders
ORDER BY Customer_ID, Order_Date;
```

Output:



7. Display top 3 customers based on total sales. (Window Function)

SQL Query:

```

WITH CustomerSales AS
(
  SELECT
    c.Customer_ID,
    c.Customer_Name,
    SUM(sr.Sales) AS Total_Sales
  FROM customers c
  JOIN orders o
    ON c.Customer_ID = o.Customer_ID
  JOIN superstore_raw sr
    ON o.Order_ID = sr.`Order ID`
  GROUP BY
    c.Customer_ID,
    c.Customer_Name
),
RankedCustomers AS
(
  SELECT
    Customer_ID,
    Customer_Name,
    Total_Sales,
    RANK() OVER (ORDER BY Total_Sales DESC) AS Customer_Rank
  FROM CustomerSales
)

SELECT

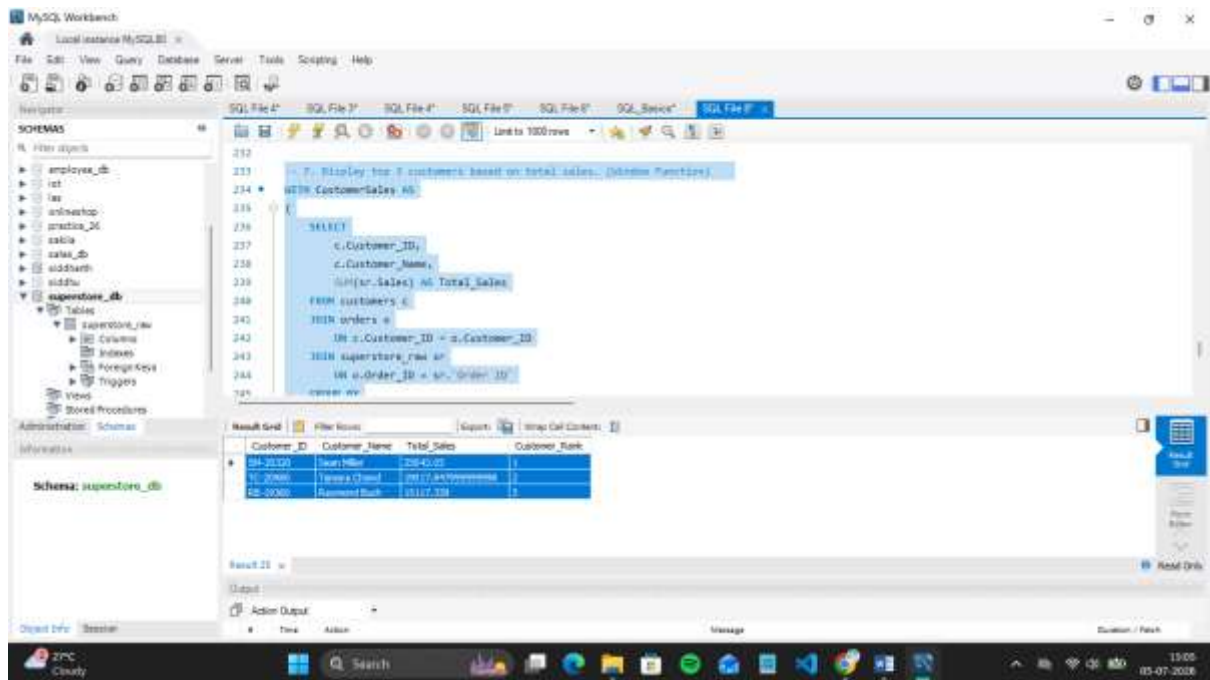
```

```

Customer_ID,
Customer_Name,
Total_Sales,
Customer_Rank
FROM RankedCustomers
WHERE Customer_Rank <= 3
ORDER BY Customer_Rank;

```

Output:



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'superstore_db' selected. The main editor window contains the following SQL query:

```

-- 7. Display top 3 customers based on total sales. (Window Function)
WITH CustomerSales AS
(
    SELECT
        c.Customer_ID,
        c.Customer_Name,
        sum(o.Sales) AS Total_Sales
    FROM customers c
    JOIN orders o
    ON c.Customer_ID = o.Customer_ID
    JOIN superstore_raw sr
    ON o.Order_ID = sr.Order_ID
)

```

The 'Result Grid' at the bottom displays the output of the query, showing the top 3 customers by total sales:

Customer_ID	Customer_Name	Total_Sales	Customer_Rank
94-20120	SeoulMart	22945.95	1
10-20686	Tamara (Toni)	2017.8476999999999	2
62-00360	Rammond Bush	15117.324	3

The bottom status bar shows the system clock as 15:05 on 05-07-2024.

Step 3: Final Combined Query

Write **one final query** that shows:

- Customer Name
- Total Sales
- Rank

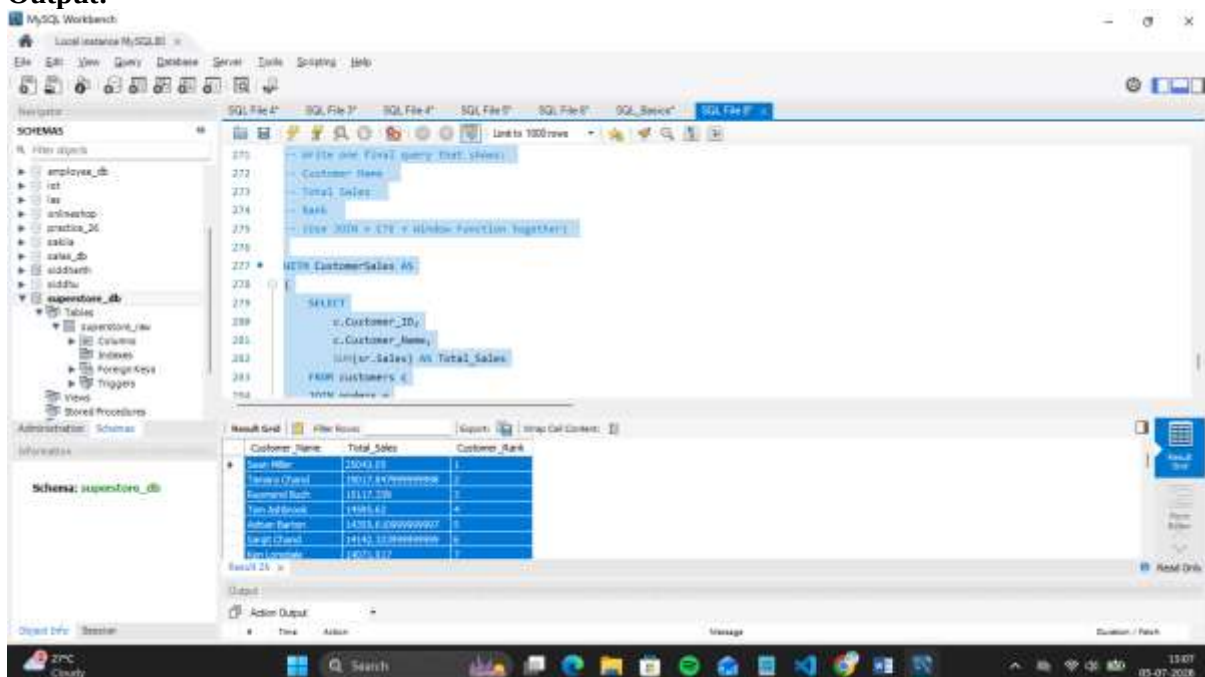
(Use **JOIN** + **CTE** + **Window Function** together)

SQL Query :

```
WITH CustomerSales AS
(
    SELECT
        c.Customer_ID,
        c.Customer_Name,
        SUM(sr.Sales) AS Total_Sales
    FROM customers c
    JOIN orders o
        ON c.Customer_ID = o.Customer_ID
    JOIN superstore_raw sr
        ON o.Order_ID = sr.`Order ID`
    GROUP BY
        c.Customer_ID,
        c.Customer_Name
)

SELECT
    Customer_Name,
    Total_Sales,
    RANK() OVER (ORDER BY Total_Sales DESC) AS Customer_Rank
FROM CustomerSales
ORDER BY Customer_Rank;
```

Output:



The screenshot shows the MySQL Workbench interface. The SQL editor contains the query defined in the previous block. The 'Result Grid' tab is active, displaying the output of the query. The output is a table with three columns: Customer Name, Total Sales, and Customer Rank. The results are ordered by Customer Rank in descending order.

Customer Name	Total Sales	Customer Rank
Sean Miller	22003.89	1
James Ozard	19017.84	2
Customer Rank	18111.728	3
Timothy Jones	14083.45	4
Adam Barker	14038.6	5
Luigi Zandi	14142.12	6
Nick Lombardi	14075.827	7

Mini Project: Customer Sales Insights

Answer the following using SQL:

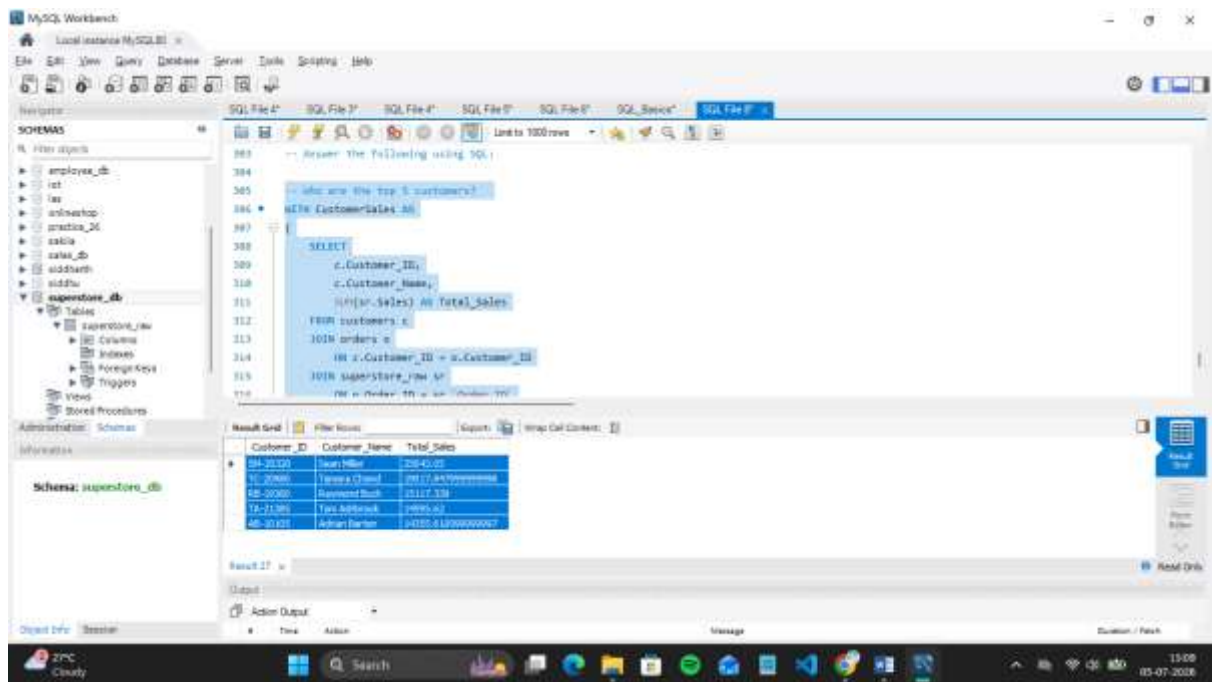
1. Who are the top 5 customers?

SQL Query:

```
WITH CustomerSales AS
(
    SELECT
        c.Customer_ID,
        c.Customer_Name,
        SUM(sr.Sales) AS Total_Sales
    FROM customers c
    JOIN orders o
        ON c.Customer_ID = o.Customer_ID
    JOIN superstore_raw sr
        ON o.Order_ID = sr.`Order ID`
    GROUP BY
        c.Customer_ID,
        c.Customer_Name
)

SELECT
    Customer_ID,
    Customer_Name,
    Total_Sales
FROM CustomerSales
ORDER BY Total_Sales DESC
LIMIT 5;
```

Output:



The screenshot shows the MySQL Workbench interface. The SQL editor contains the query to find the top 5 customers by total sales. The results are displayed in a table with columns Customer_ID, Customer_Name, and Total_Sales.

Customer_ID	Customer_Name	Total_Sales
104-20-000	Scars Hill	25945.00
10-20-000	Tamara (Tami)	20177.84
40-20-000	Raymond Bush	15117.33
16-21-000	Tara Adkins	14895.63
46-30-000	Adam Barton	14055.63

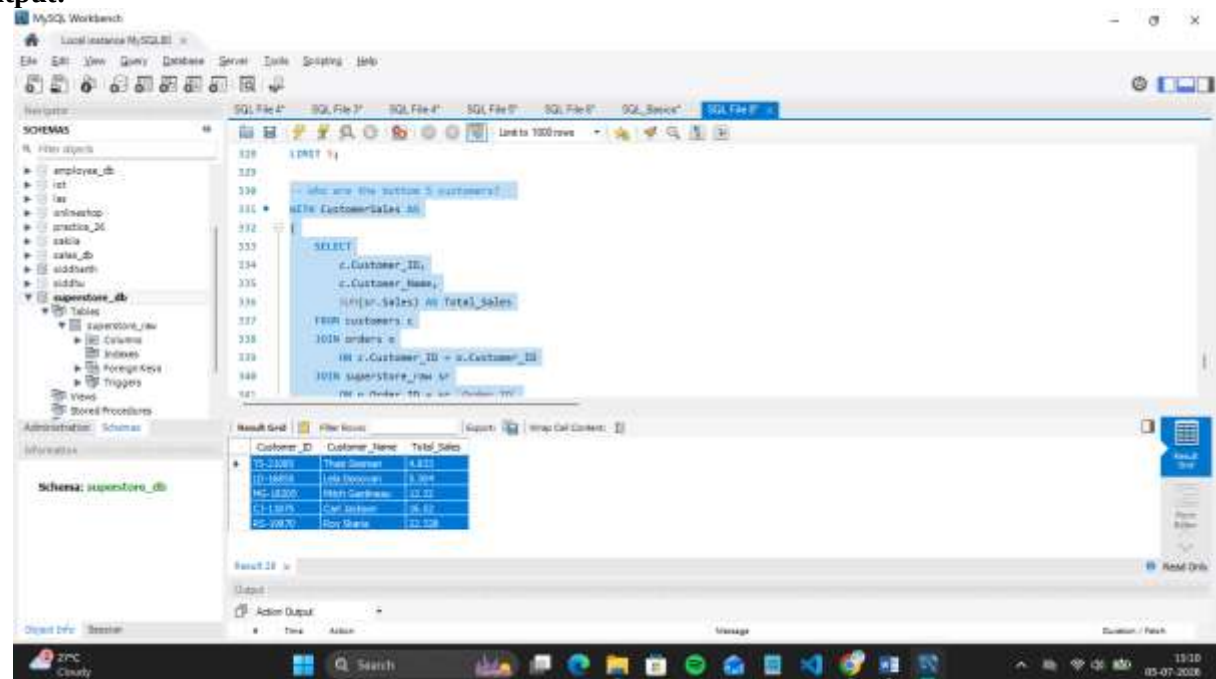
2. Who are the bottom 5 customers?

SQL Query:

```
WITH CustomerSales AS
(
    SELECT
        c.Customer_ID,
        c.Customer_Name,
        SUM(sr.Sales) AS Total_Sales
    FROM customers c
    JOIN orders o
        ON c.Customer_ID = o.Customer_ID
    JOIN superstore_raw sr
        ON o.Order_ID = sr.`Order ID`
    GROUP BY
        c.Customer_ID,
        c.Customer_Name
)

SELECT
    Customer_ID,
    Customer_Name,
    Total_Sales
FROM CustomerSales
ORDER BY Total_Sales ASC
LIMIT 5;
```

Output:



The screenshot shows the MySQL Workbench interface. The SQL editor contains the query to find the bottom 5 customers by total sales. The output is displayed in the Results Grid, showing the top 5 customers with their IDs, names, and total sales.

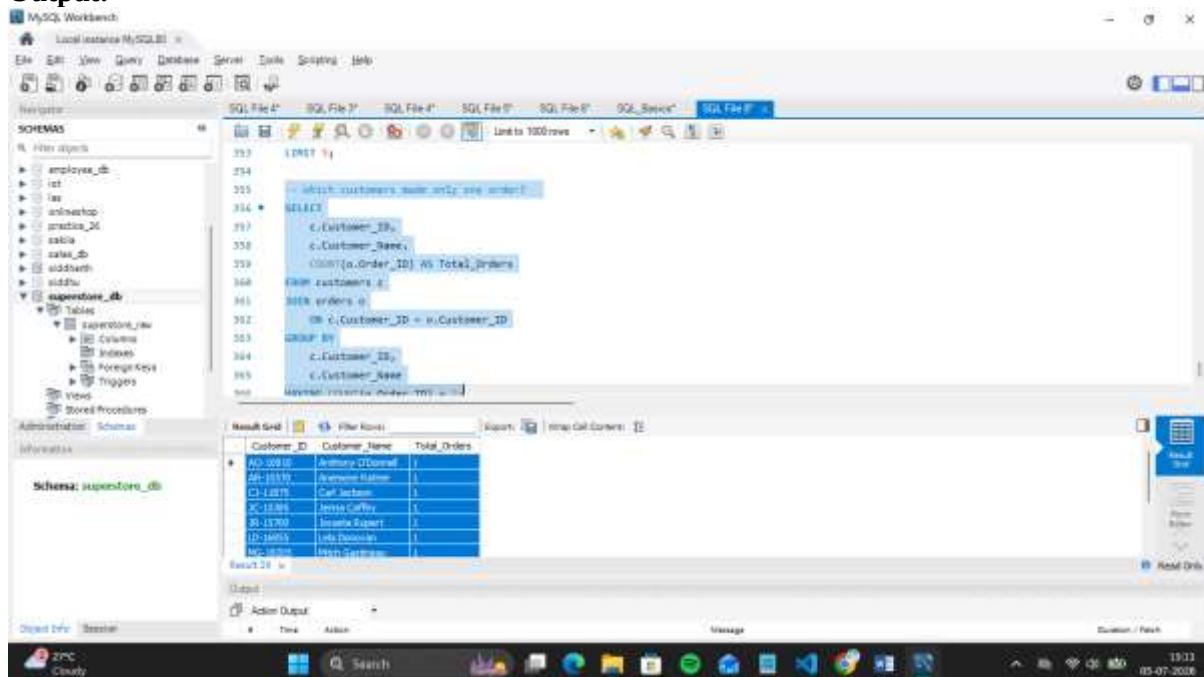
Customer_ID	Customer_Name	Total_Sales
TS-23289	Thad Stearns	1,523
TD-16898	John Woodward	1,394
MG-18200	Mark Greenman	13.22
CL-15876	Carl Jackson	26.82
RG-99870	Ray Steele	23.528

3. Which customers made only one order?

SQL Query:

```
SELECT
    c.Customer_ID,
    c.Customer_Name,
    COUNT(o.Order_ID) AS Total_Orders
FROM customers c
JOIN orders o
    ON c.Customer_ID = o.Customer_ID
GROUP BY
    c.Customer_ID,
    c.Customer_Name
HAVING COUNT(o.Order_ID) = 1;
```

Output:



The screenshot shows the MySQL Workbench interface. The SQL editor contains the query for customers with only one order. The 'Result Grid' tab is active, displaying the results of the query. The output table has three columns: Customer_ID, Customer_Name, and Total_Orders. The results show six customers, each with a Total_Orders value of 1.

Customer_ID	Customer_Name	Total_Orders
AD-0816	Anthony D'Amore	1
AF-1510	Alexander Fattori	1
CH-1819	Chai Jackson	1
SC-1505	Shirley Corley	1
SS-1370	Stefano Expert	1
TD-1905	Timothy Baerlin	1

4. Which customers have above-average sales?

SQL Query:

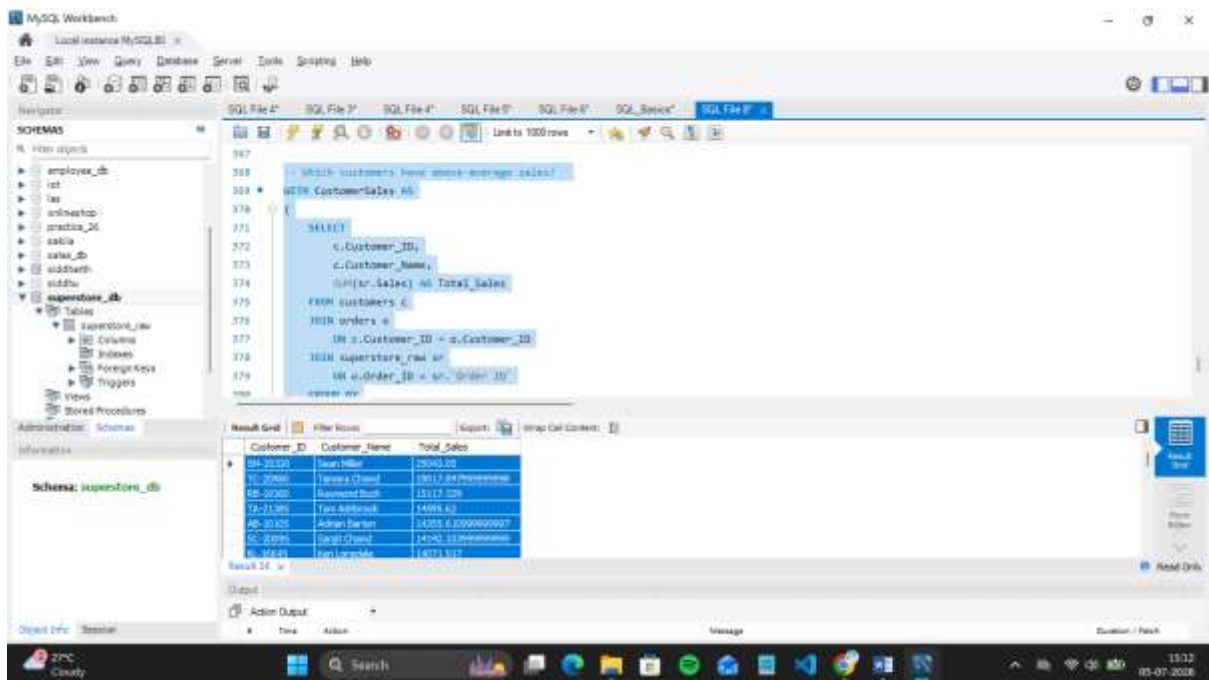
```
WITH CustomerSales AS
(
    SELECT
        c.Customer_ID,
        c.Customer_Name,
        SUM(sr.Sales) AS Total_Sales
    FROM customers c
    JOIN orders o
        ON c.Customer_ID = o.Customer_ID
    JOIN superstore_raw sr
        ON o.Order_ID = sr.`Order ID`
    GROUP BY
        c.Customer_ID,
        c.Customer_Name
)
```

```

SELECT
    Customer_ID,
    Customer_Name,
    Total_Sales
FROM CustomerSales
WHERE Total_Sales >
(
    SELECT AVG(Total_Sales)
    FROM CustomerSales
)
ORDER BY Total_Sales DESC;

```

Output:



5. What is the highest order value per customer?

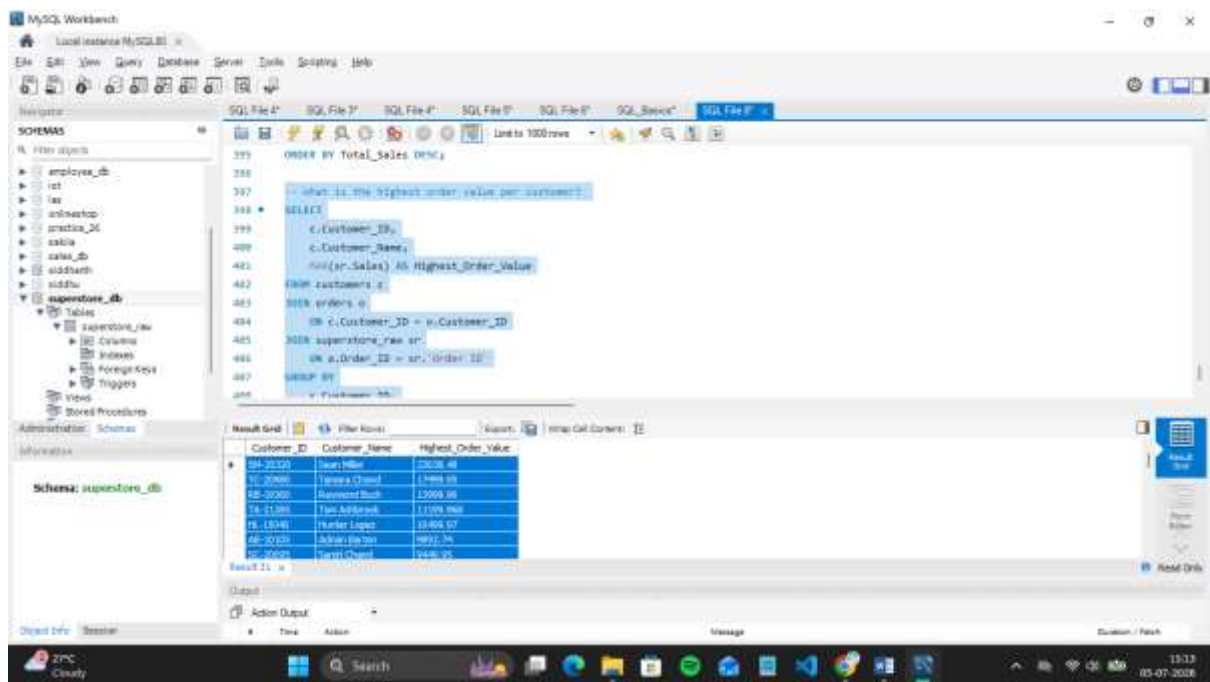
SQL Query:

```

SELECT
    c.Customer_ID,
    c.Customer_Name,
    MAX(sr.Sales) AS Highest_Order_Value
FROM customers c
JOIN orders o
    ON c.Customer_ID = o.Customer_ID
JOIN superstore_raw sr
    ON o.Order_ID = sr.`Order ID`
GROUP BY
    c.Customer_ID,
    c.Customer_Name
ORDER BY Highest_Order_Value DESC;

```

Output:



The screenshot displays the MySQL Workbench interface. On the left, the 'SCHEMAS' pane shows a list of databases, with 'supernet_db' selected. The main editor window contains a SQL query designed to find the highest order value for each customer. The query uses a subquery to determine the maximum order value for each customer and then joins this with the 'supernet_orders' table to retrieve the corresponding customer names.

```
335 ORDER BY Total_Sales Desc;
336
337 -- what is the highest order value per customer?
338
339 SELECT
340     c.Customer_ID,
341     c.Customer_Name,
342     max(o.Total_Sales) AS Highest_Order_Value
343 FROM customers c
344 JOIN orders o
345     ON c.Customer_ID = o.Customer_ID
346 JOIN supernet_new sup
347     ON o.Order_ID = sup.Order_ID
348 GROUP BY
349     c.Customer_ID;
```

Below the query editor, the 'Result Grid' shows the output of the query. It contains 6 rows of data, each representing a customer and their highest order value.

Customer_ID	Customer_Name	Highest_Order_Value
10-20000	Sarah Miller	22036.48
10-20001	Tamara Christ	14488.35
10-20002	Raymond Buck	12008.95
10-21000	Tina Anderson	11009.95
10-10000	Hunter Lopez	10405.07
10-20003	Adrian Nelson	9892.75

The bottom of the screen shows the Windows taskbar with the date and time as 03-07-2028 15:13.